



Authorized Web Application Security Assessment

Project 1 - OWASP Juice Shop Web and API Security Validation

Target: OWASP Juice Shop - intentionally vulnerable training lab

Prepared By: Anyaugo Space-Tech LTD

Assessment Type: Web Application and API Security Assessment

Methodology: OWASP Web Security Testing Guide and OWASP Top 10

Document Classification: Public Portfolio Sample / Non-Client Demonstration

Prepared For: Portfolio Review / Technical Assessment

Date: September 14, 2026

Evidence Screenshots: 9

Assessment Environment Note

This portfolio demonstration was completed in a controlled local lab using OWASP Juice Shop, an intentionally vulnerable open-source web application created for legal security training.

Portfolio Boundary

This document does not represent an assessment of a real-world enterprise system. It demonstrates methodology, evidence collection, risk analysis, business impact communication, remediation guidance, and retest planning.

Project 1 Main Topic

Authorized web application security assessment showing authentication, access-control, client-side injection, API exposure, and object-level authorization risks using a safe training target.

Executive Summary

An unauthenticated security assessment was performed against the OWASP Juice Shop lab application. The objectives were to evaluate the attack surface, identify representative weaknesses, explain business impact, and provide actionable remediation guidance.

The assessment identified five representative findings across authentication, access control, input handling, and configuration management. In a real environment, similar weaknesses could create risk around account access, sensitive files, client-side script execution, API discovery, and non-owned customer resources.

Findings Summary

ID	Finding	Severity	Mapping
Finding 1	SQL Injection - Authentication Bypass	Critical	OWASP Top 10 A05 - Injection WSTG-INPV-05
Finding 2	Sensitive Data Exposure via Directory Listing	High	OWASP Top 10 A01 - Broken Access Control WSTG-CONF-04
Finding 3	DOM-Based Cross-Site Scripting	High	OWASP Top 10 A05 - Injection WSTG-CLNT-01
Finding 4	Information Disclosure via Exposed API Documentation	Medium	OWASP Top 10 A02 - Security Misconfiguration WSTG-CONF-05
Finding 5	Insecure Direct Object Reference on User Baskets	High	OWASP Top 10 A01 - Broken Access Control WSTG-ATHZ-04

Scope And Methodology

- Application target: OWASP Juice Shop local lab.
- Scope surface: public web frontend, REST API behavior, static file endpoints, and browser-side storage mechanisms.
- Evidence method: browser-based manual validation, developer tools review, request/response inspection, and screenshot capture.
- Scoring method: baseline severity estimates using CVSS concepts plus business impact. Production scores should be calibrated to the client's environment.

Tools And Delivery Approach

OWASP Juice Shop	Intentionally vulnerable local web application lab.
Browser Developer Tools	Network, Console, Application Storage, and XHR/Fetch inspection.
Manual Web Testing	Authenticated and unauthenticated request validation.
Screenshot Evidence	Finding documentation and proof capture.
Report Structuring	Business impact wording, remediation guidance, retest planning, and portfolio review.
Commercial Engagement Rule	In a real client engagement, this work must be performed only after written authorization, approved scope, documented testing windows, and clear stop conditions. The portfolio evidence demonstrates method, not permission to test real organizations.

Finding 1: SQL Injection - Authentication Bypass

Severity: Critical (9.8 baseline estimate)

Taxonomy: OWASP Top 10 A05 - Injection | WSTG-INPV-05

Description

The authentication endpoint in the lab failed to safely handle user-supplied input before database query evaluation, allowing authentication logic to be bypassed in a controlled training environment.

Business Impact

Could allow an unauthenticated actor to bypass login and access privileged account functions if similar behavior existed in production.

Remediation Guidance

Use parameterized queries or ORM prepared statements, validate input server-side, and store session identifiers in HttpOnly and Secure cookies instead of browser-accessible local storage.

Evidence

Finding1A-Empty-Local-Storage.png, Finding1B-Populated-Token-In-Local-Storage.png, Finding1C-XHR-Request.png

Finding 2: Sensitive Data Exposure via Directory Listing

Severity: High (7.5 baseline estimate)

Taxonomy: OWASP Top 10 A01 - Broken Access Control | WSTG-CONF-04

Description

Directory indexing was enabled on the public lab endpoint, allowing unauthenticated users to inspect directory contents and access internal documents.

Business Impact

Could expose sensitive documentation and accelerate reconnaissance against a real application.

Remediation Guidance

Disable directory browsing, remove sensitive files from public paths, and enforce server-side access control for static assets that should not be public.

Evidence

Finding2A-FTP-Directory.png, Finding2B-Exfiltrated-Confidential-Document.png

Finding 3: DOM-Based Cross-Site Scripting

Severity: High (7.2 baseline estimate)

Taxonomy: OWASP Top 10 A05 - Injection | WSTG-CLNT-01

Description

The search functionality reflected user-controlled input into the browser DOM without sufficient output encoding, allowing script execution in the controlled lab environment.

Business Impact

Could support session manipulation, interface deception, or arbitrary client-side request execution when combined with other weaknesses.

Remediation Guidance

Apply context-aware output encoding, avoid unsafe DOM sinks, sanitize user-controlled input before rendering, and enforce a restrictive Content Security Policy.

Evidence

Finding3-XSS-Execution.png

Finding 4: Information Disclosure via Exposed API Documentation

Severity: Medium (5.3 baseline estimate)

Taxonomy: OWASP Top 10 A02 - Security Misconfiguration | WSTG-CONF-05

Description

The application exposed interactive Swagger/OpenAPI documentation without authentication in the lab environment.

Business Impact

Public API documentation can reveal endpoints, naming conventions, payload structures, and possible attack surfaces.

Remediation Guidance

Disable Swagger UI and schema exposure in production or restrict access with authentication, authorization, and environment-specific configuration.

Evidence

Finding4A-Swagger-Interface.png, Finding4B-Exposed-API-Schema.png

Finding 5: Insecure Direct Object Reference on User Baskets

Severity: High (8.5 baseline estimate)

Taxonomy: OWASP Top 10 A01 - Broken Access Control | WSTG-ATHZ-04

Description

The basket API returned resource data based on a requested identifier without adequately confirming that the requesting user owned the resource.

Business Impact

Could allow unauthorized access to private customer records or order activity across non-owned application accounts.

Remediation Guidance

Implement object-level authorization checks on every request, verify ownership server-side, and add regression tests for horizontal access-control failures.

Evidence

Finding5-IDOR-Basket-Exfiltration.png

Remediation Verification And Retest Plan

For this portfolio demonstration, the retest plan describes how each identified vulnerability would be validated after remediation in the controlled OWASP Juice Shop lab environment.

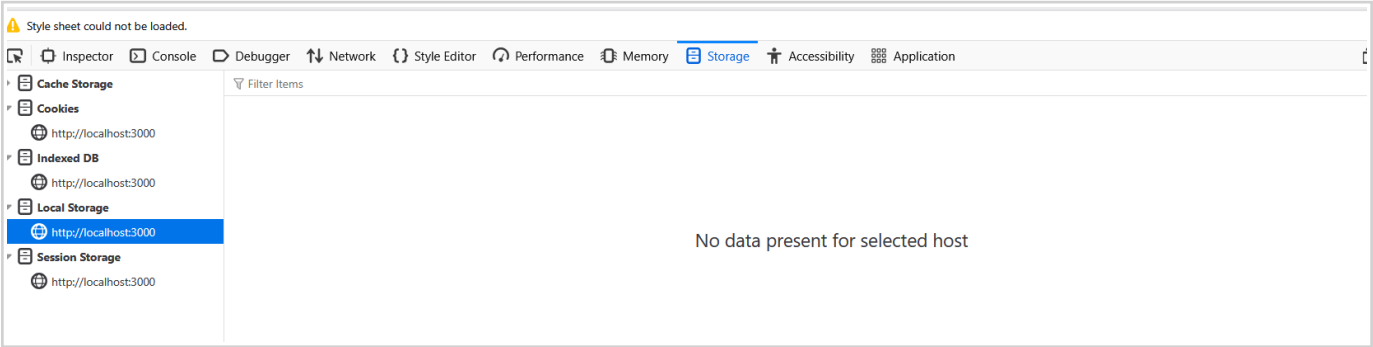
SQL Injection	Recheck the authentication endpoint and confirm the bypass condition no longer succeeds.
Directory Listing	Access the exposed path and confirm browsing is disabled or restricted.
DOM-Based XSS	Resubmit the test input and confirm it is encoded and rendered safely as text.
API Documentation Exposure	Query the documentation path and confirm production access is disabled or restricted.
IDOR	Request a non-owned basket resource and confirm the API rejects the request.

Client Engagement Retest Rules

Retesting in a commercial engagement is performed exclusively on authorized client assets after formal written authorization, agreed scope, and confirmation that fixes have been deployed. The final deliverable is a remediation verification addendum.

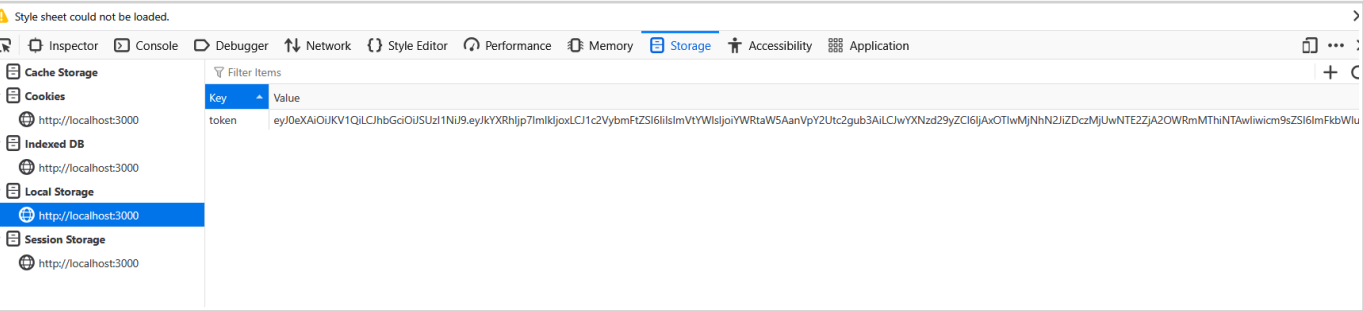
Evidence A1 - Empty browser local storage before authentication.

Source file: Test-1-SQL-Injection\Finding1A-Empty-Local-Storage.png



Evidence A2 - Token appears in browser storage after the lab authentication

Source file: Test-1-SQL-Injection\Finding1B-Populated-Token-In-Local-Storage.png



Evidence A3 - XHR request and authentication response context.

Source file: Test-1-SQL-Injection\Finding1C-XHR-Request.png

Inspector

Console

Debugger

Network

Style Editor

Performance

Memory

Storage

Accessibility

Application

Filter URLs

Status	Method	Domain	File	Initiator	Type	Transferred	Size
200	GET	localhost:3000	/socket.io/?EIO=4&transport=polling&tt=Q251K27	polyfills.js:1 (xhr)	plain	326 B	96 B
200	GET	localhost:3000	en.json	polyfills.js:2 (fetch)	json	13.54 kB	35.47 kB
200	GET	localhost:3000	application-version	polyfills.js:2 (fetch)	json	404 B	20 B
200	GET	localhost:3000	application-configuration	polyfills.js:2 (fetch)	json	9.52 kB	23.58 kB
200	GET	localhost:3000	/api/Challenges/?name=Score Board	polyfills.js:2 (fetch)	json	862 B	476 B
200	GET	localhost:3000	languages	polyfills.js:2 (fetch)	json	1.48 kB	4.22 kB
304	GET	localhost:3000	application-version	polyfills.js:2 (fetch)	json	cached	20 B
304	GET	localhost:3000	application-configuration	polyfills.js:2 (fetch)	json	cached	23.58 kB
304	GET	localhost:3000	/api/Challenges/?name=Score Board	polyfills.js:2 (fetch)	json	cached	476 B
304	GET	localhost:3000	application-configuration	polyfills.js:2 (fetch)	json	cached	23.58 kB
304	GET	localhost:3000	application-configuration	polyfills.js:2 (fetch)	json	cached	23.58 kB
200	GET	localhost:3000	/api/Quantities/	polyfills.js:2 (fetch)	json	940 B	7.63 kB
200	GET	localhost:3000	search?q=	polyfills.js:2 (fetch)	json	4.77 kB	16.56 kB
200	POST	localhost:3000	/socket.io/?EIO=4&transport=polling&tt=Q251K7b&sid=t	polyfills.js:1 (xhr)	html	215 B	2 B
200	GET	localhost:3000	/socket.io/?EIO=4&transport=polling&tt=Q251K7r&sid=C	polyfills.js:1 (xhr)	plain	262 B	32 B
200	GET	localhost:3000	/socket.io/?EIO=4&transport=polling&tt=Q251KGR&sid=	polyfills.js:1 (xhr)	plain	230 B	1 B

16 requests | 159.30 kB / 32.55 kB transferred | Finish: 4.31 s | DOMContentLoaded: 502 ms | load: 2.40 s

Headers

Cookies

Request

Response

Timings

Stack Trace

Filter properties

JSON

Raw

LANGUAGE: "English"

NAV_SEARCH: "Search"

SEARCH_PLACEHOLDER: "Search..."

NAV_COMPLAIN: "Complaint"

TITLE_LOGIN: "Login"

MANDATORY_EMAIL: "Please provide an email address."

MANDATORY_PASSWORD: "Please provide a password."

LABEL_EMAIL: "Email"

LABEL_PASSWORD: "Password"

SHOW_PASSWORD_ADVICE: "Show password advice"

LOWER_CASE_CRITERIA_MSG: "contains at least one lower character"

UPPER_CASE_CRITERIA_MSG: "contains at least one upper character"

DIGITS_CRITERIA_MSG: "contains at least one digit"

SPECIAL_CHARS_CRITERIA_MSG: "contains at least one special character"

MIN_CHARS_CRITERIA_MSG: "contains at least (lvalue) characters"

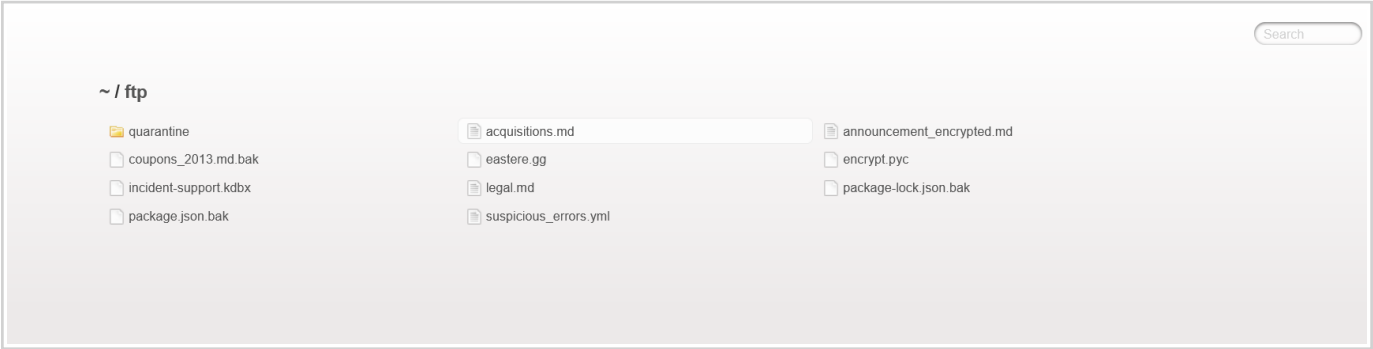
BTN_LOGIN: "Log in"

BTN_GOOGLE_LOGIN: "Log in with Google"

REMEMBER_ME: "Remember me"

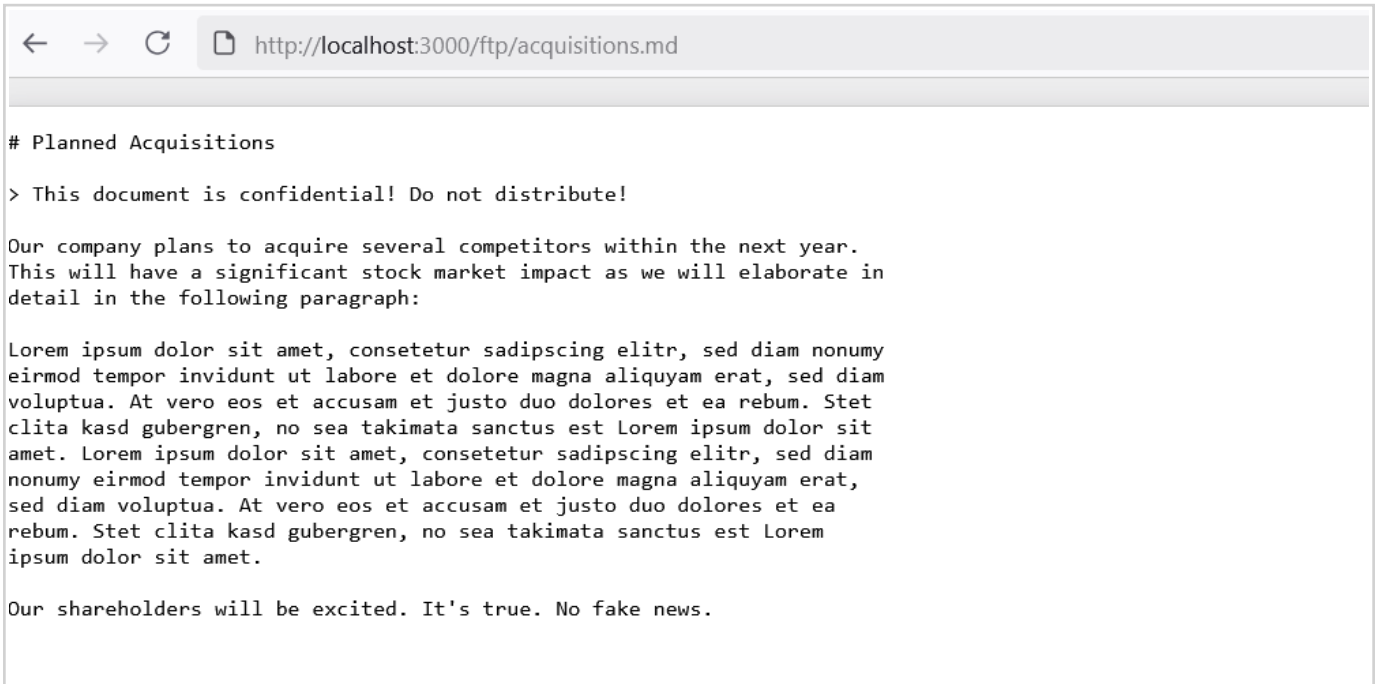
Evidence B1 - Public FTP-style directory listing in the lab.

Source file: Test-2-Broken-Access-Control_Sensitive-Data-Exposure\Finding2A-FTP-Directory.png



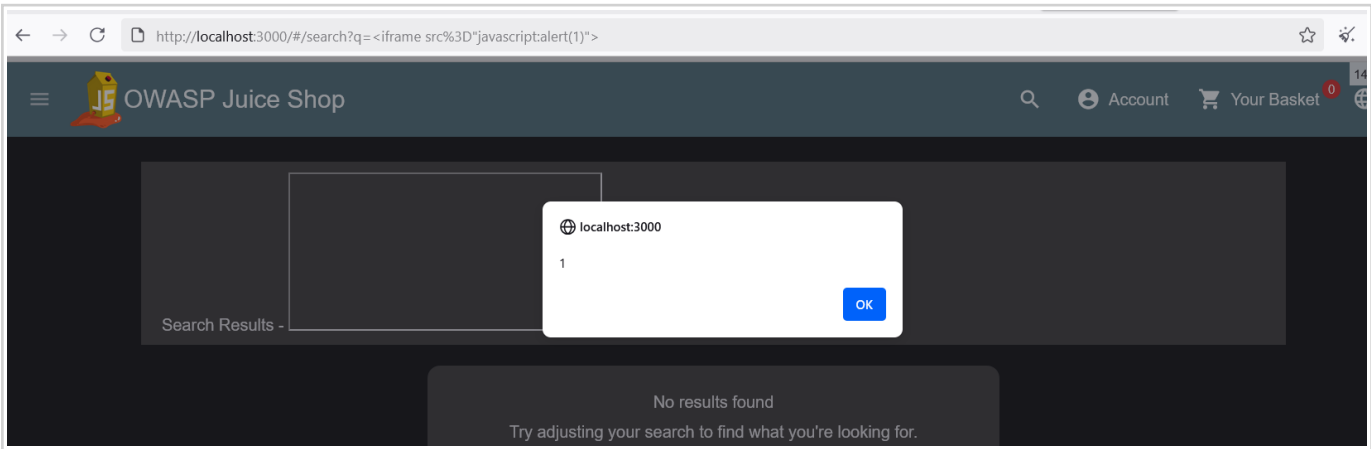
Evidence B2 - Internal document exposed through the lab directory.

Source file: Test-2-Broken-Access-Control_Sensitive-Data-Exposure\Finding2B-Exfiltrated-Confidential-Document.png



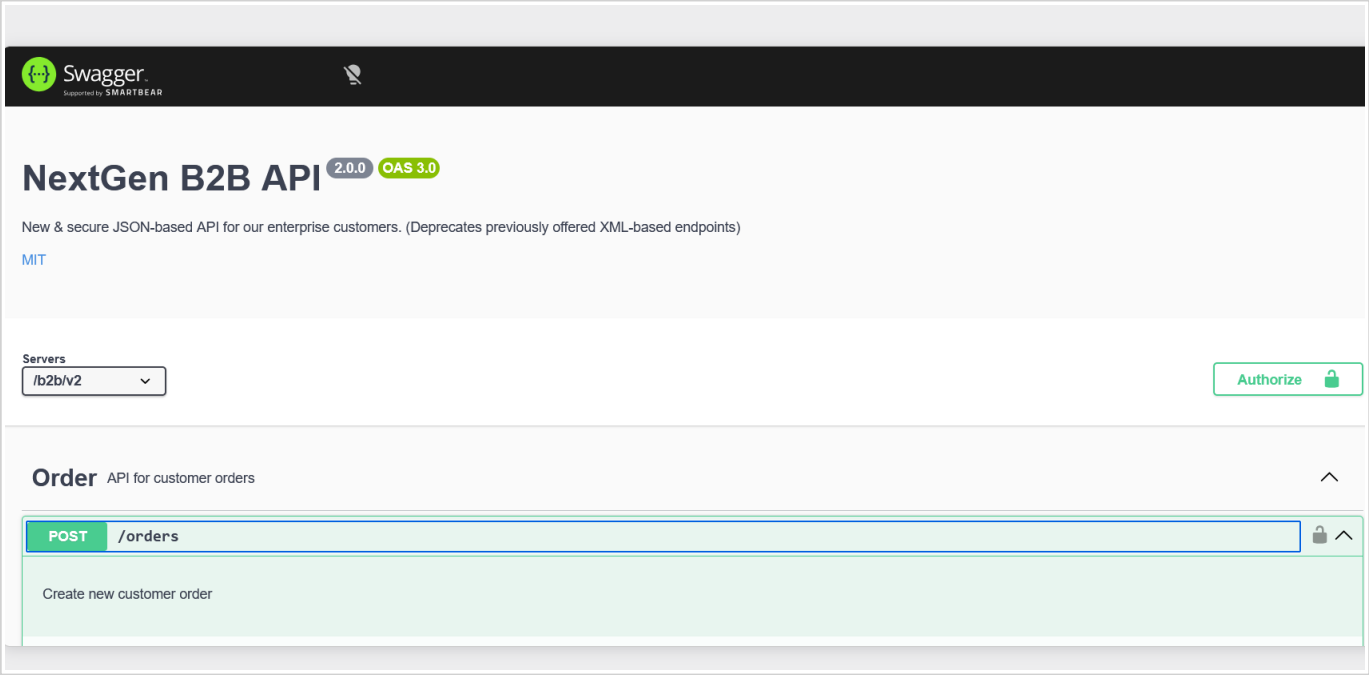
Evidence C1 - DOM-based XSS execution proof in the lab application.

Source file: Test-3-DOM-Based Reflected Cross-Site Scripting (XSS)\Finding3-XSS-Execution.png



Evidence D1 - Public Swagger/OpenAPI interface.

Source file: Test-4-Information-Disclosure-via-Exposed-Endpoint\Finding4A-Swagger-Interface.png



Evidence D2 - Exposed API schema details.

Source file: Test-4-Information-Disclosure-via-Exposed-Endpoint\Finding4B-Exposed-API-Schema.png

Request body

application/json

Customer order to be placed

Example Value | Schema

```
{
  "cid": "J58815DE",
  "orderLines": [
    {
      "productId": 8,
      "quantity": 500,
      "customerReference": "P00000001"
    }
  ],
  "orderLinesData": "[[{\\"productId\\": 12,\\\"quantity\\": 10000,\\\"customerReference\\": [\\\"P00000001.2\\\", \\\"SM20180105[042\\\",\\\"couponCode\\": \\\"pes[Bh.u*t\\\",{\\"productId\\": 13,\\\"quantity\\": 2000,\\\"customerReference\\": \\\"P00000003.4\\\"}]]]"
}
```

Responses

Code	Description	Links
200	New customer order is created	No links

Media type

application/json

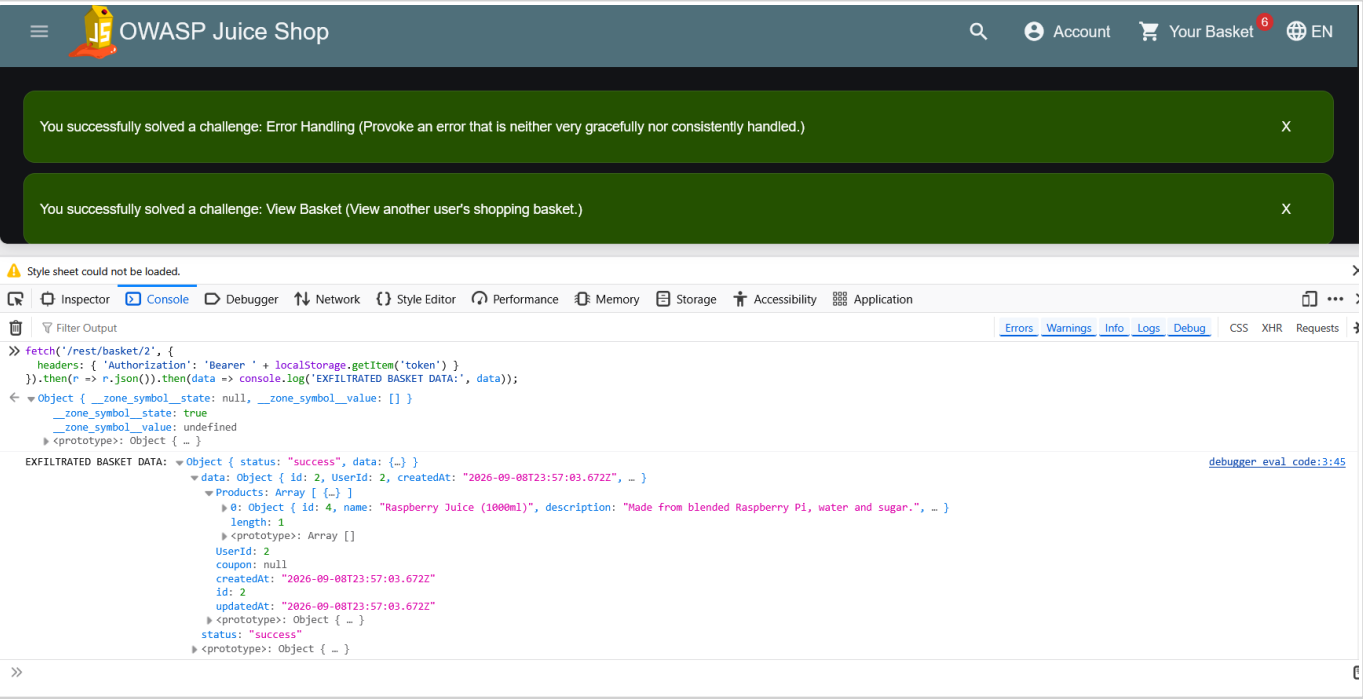
Controls Accept header.

Example Value | Schema

```
{
  "cid": "J58815DE",
  "orderNo": "3d065c5e1bdf39d26392f8100f124742",
  "paymentDue": "2018-01-19"
}
```

Evidence E1 - Basket data returned for a non-owned resource in the lab.

Source file: Test-5-Insecure-Direct-Object-Reference-(IDOR) _Broken-Object-Level-Authorization\Finding5-IDOR-Basket-Exfiltration.png



References

- OWASP Juice Shop - <https://owasp.org/www-project-juice-shop/>
- OWASP Web Security Testing Guide - <https://owasp.org/www-project-web-security-testing-guide/>
- OWASP Top 10 - <https://owasp.org/Top10/>
- FIRST CVSS - <https://www.first.org/cvss/>